

Rapport d'avancement — Espace Organisateur

Stagiaire : Lorenzo BUONOCORE **Période** : 18 mai -> 19 juin 2026 **Projet** : Développement de la logique de l'Espace Organisateur

Contexte général

Développement de l'**Espace Organisateur V3** — interface permettant aux organisateurs de créer, configurer et gérer leurs épreuves.

Stack technique : PHP (backend/vues), JavaScript vanilla, MySQL/PDO, Tailwind CSS, Quill.js

Bibliothèque globale — `lib_esporg__global.php`

- Création de fonctions de lecture BDD : événements, épreuves, participants, tarifs, extras, codes promo, remboursements, documents à vérifier, infos organisateur
 - Fonctions dédiées à l'équipe d'une epreuve : `EspOrgGlobal__Donner_Equipe_Info`, `EspOrgGlobal__Donner_Equipe_Ctg`, `EspOrgGlobal__Donner_Operateurs_Liste`
 - Fonctions utilitaires : `EspOrgGlobal__Retirer_Accent`, `EspOrgGlobal__Formater_Date_Pour_Affichage`, `EspOrgGlobal__Nettoyer_Nom_Ville`, `EspOrgGlobal__Obtenir_Info_Ou_Actu_La_Plus_Recente`
-

Bibliothèque statistiques — `lib_esporg__stats.php`

Bibliothèque dédiée aux statistiques, créée de zéro :

- `EspOrgStats__Infos_Evenement_A_Date_Equivalente` — compare les inscrits à une même période d'avance entre deux éditions
- `EspOrgStats__Comparatif_Infos_Evenement_Par_Anee` — comparatif multi-éditions (inscrits, gains KMS)
- `EspOrgStats__Historique_Inscription_Par_Edition` — courbe historique de toutes les éditions d'un événement
- `EspOrgStats__Nb_Nouvelles_Inscriptions_Evenement_Jour_J` — compteur en temps réel des inscriptions du jour
- `EspOrgStats__Nb_Candidat_Sexe_Par_Ctg` — répartition hommes/femmes par catégorie
- `EspOrgStats__Inscriptions_Par_Semaine_Avant_Course` — courbe d'inscriptions par semaine (jusqu'à N mois avant la course)

- `EspOrgStats__Inscriptions_Par_Departement` — distribution géographique par département
 - `EspOrgStats__Inscriptions_Par_Ville` — distribution par ville avec coordonnées GPS (pour la carte)
 - `EspOrgStats__Candidat_Le_Plus_Ou_Moins_Agee` — participant le plus jeune / le plus âgé
 - `EspOrgStats__Candidats_Mineurs_Par_Ctg` — comptage des mineurs par catégorie
 - `EspOrgStats__Utilisation_Promo_Par_Ctg` — utilisation des codes promo par catégorie
 - `EspOrgStats__Reponses_Extras_Par_Ctg` — réponses aux extras par catégorie
 - `EspOrgStats__Obtenir_Nb_Editions` — nombre d'éditions d'un événement
-

Bibliothèque actions — `lib_esporg__actions.php`

- `EspOrgActions__Ajout_Course` : création d'une nouvelle course en BDD
 - `EspOrgActions__MaJ_Infos_Course_Step1` : événement, type, nom, date, région, département, ville, distance
 - `EspOrgActions__MaJ_Infos_Course_Step2` : dates d'inscription, places max, état, raison d'arrêt, remboursement
 - `EspOrgActions__MaJ_Infos_Course_Step3` : contraintes sexe/âge, navettes, vestiaires, handisport + **bloc équipe complet** (nb équipiers, relais, rôles, catégories, infos équipiers, responsable)
 - `EspOrgActions__Verifier_Remboursement_Existe` : helper interne qui crée la ligne de remboursement si elle n'existe pas encore
-

Bibliothèque communication — `lib_esporg__communication.php`

Bibliothèque dédiée au système de notifications et de messagerie, créée de zéro. Elle s'appuie sur les tables `notifications_v2`, `notifications_v2_destinataires` et `notifications_v2_config`.

Génération automatique des notifications

Quatre fonctions créent ou mettent à jour automatiquement les notifications au chargement de la page, selon les données en temps réel :

- `EspOrgCommu__Ajouter_Notification_Doc_Verif($id_org)` — parcourt tous les événements à venir de l'organisateur, compte les documents à vérifier via `EspOrgGlobal__Donner_Doc_A_Verif_Liste`, génère une notif de type `[DOC] - Evn #X` avec le nombre de documents en attente
- `EspOrgCommu__Ajouter_Notification_Dmd_Remboursement($id_org)` — même logique pour les demandes de remboursement en attente (`[RMB] - Evn #X`)

- `EspOrgCommu__Ajouter_Notification_Profil($id_org)` — détecte si le profil organisateur est incomplet pour un événement à venir (`[PROFIL] - Evn #X`)
- `EspOrgCommu__Ajouter_Notification_Course_Complete($id_org)` — vérifie si `nb_inscrits >= nb_max` pour chaque épreuve et génère une notif `[COMPLETE] - Crs #X`

Chacune de ces fonctions suit le même pattern : **INSERT si nouvelle, UPDATE si existante** (via `EspOrgCommu__Verif_Notif_Existante`), puis lie la notification à l'organisateur dans `notifications_v2_destinataires` .

- `EspOrgCommu__Ajouter_Notification($id_org)` — **fonction maître** qui appelle les 4 fonctions ci-dessus en une seule fois ; c'est elle qui est appelée au chargement de `EspOrgCommu__Donner_Notifications_Liste`

Helpers internes

- `EspOrgCommu__Verif_Notif_Existante($id_event, $annee_event, $titre)` — vérifie si une notification identique (même événement, même édition, même titre) existe déjà en BDD pour éviter les doublons
- `EspOrgCommu__Verif_Notif_Destinataire($id_org, $idNotif)` — vérifie si la ligne `notifications_v2_destinataires` existe pour cet organisateur, la crée sinon ; crée aussi la ligne `notifications_v2_config` si elle est absente

Lecture et affichage

- `EspOrgCommu__Donner_Notifications_Liste($id_org, $id_event, $annee_event, $que_non_lu)` — retourne toutes les notifications de l'organisateur (filtrables par événement/édition et par état lu/non lu) ; déclenche automatiquement `EspOrgCommu__Ajouter_Notification` avant la lecture pour avoir des données fraîches
- `EspOrgCommu__Gestion_Icon_Notifications($type)` — retourne le tableau `[color, icon]` Iconify selon le type : `success` (vert), `warning` (orange), `info` (violet)

Marquer comme lu — logique délégué

`EspOrgCommu__Marquer_Notif_Lu($id_mbr, $id_notif)` implémente une logique avancée à deux modes :

- **Mode 0** (défaut) : seul l'utilisateur connecté marque ses propres notifications comme lues
- **Mode 1** : si c'est l'organisateur principal qui lit, toutes les notifications de ses délégués sont également marquées lues (propagation)
- La détection délégué/organisateur se fait via une jointure sur la table `delegates`
- Si `$id_notif = 0` → toutes les notifications non lues sont marquées ; sinon une seule

Configuration du mode de lecture

- `EspOrgCommu__Donner_Mode_Lecture_Notif($id_mbr)` — retourne le mode actuel (0 ou 1) depuis `notifications_v2_config`
- `EspOrgCommu__MaJ_Mode_Lecture_Notif($id_mbr, $mode_lecture)` — met à jour le mode, avec vérification que le membre est bien de groupe 2 (organisateur)

Messagerie — `EspOrgCommu__Donner_Messages_Liste`

- Inclut un **mode test** avec 3 messages fictifs (Hervé, Romain, Bidon) (⌚ A remplacer..)

Intégration avec `lorenzo.js` et la navbar

La bulle de notification dans la navbar (`#bulle-notif`) est décrémentée en JS via `LireNotif(idmbr, idnotif)` qui appelle `read-notif.php` en AJAX — ce endpoint côté PHP appelle `EspOrgCommu__Marquer_Notif_Lu` et retourne `true` / `false`. Le compteur est mis à jour visuellement sans rechargement de page.

Formulaire multi-étapes — Création/modification d'une course

Étape	Fichier	État
1 — Présentation	<code>course_add_step1.php</code>	✓ Complet
2 — Inscriptions	<code>course_add_step2.php</code>	✓ Complet
3 — Infos complémentaires + Équipe	<code>course_add_step3.php</code>	✓ Complet
4 — Options médicales	<code>course_add_step4.php</code>	⌚ À faire
5 — Tarifs	<code>course_add_step5.php</code>	⌚ À faire
6 — Extras	<code>course_add_step6.php</code>	⌚ À faire
7 — Code promo	<code>course_add_step7.php</code>	⌚ À faire

JavaScript — Fonctions créées

- `ajouterEquipier(nb)` — régénère les inputs rôles dynamiquement
- `afficherRole()` — toggle le bloc rôles selon sélection relais
- `verifierNbEquipier()` — validation du nombre d'équipiers avec feedback visuel
- `afficherCtgMasc/Femi/Mixte/Entreprise()` — toggle les configs de chaque catégorie
- `ajouterCtg(nom)` — ajout dynamique de catégorie custom avec contrôle doublon
- `supprimerCtg(id, checkbox, nom)` — suppression avec confirmation
- `afficherInfosResp()` — toggle options responsable d'équipe

- `afficherDistance()`, `afficherNavette()`, `afficherVestiaire()`, `afficherHandicap()` — toggles des sections à précision Quill

Pages AJAX — Endpoints JSON

Fichier	Rôle
<code>stats_sexe.php</code>	Retourne la répartition H/F par catégorie en JSON, filtrable par événement ou course
<code>stats_semaine.php</code>	Retourne le nombre d'inscriptions par semaine avant la course en JSON
<code>get_coords.php</code> / <code>get_departements.php</code>	Retourne les coordonnées GPS et les départements par région pour les selects dynamiques et la carte MapLibre

Tous ces endpoints sont appelés en `fetch()` asynchrone depuis le JS, retournent du JSON et sont sécurisés par vérification de session.

Module Statistiques — `stats.js`

Fichier JS dédié aux graphiques et à la carte, avec appels AJAX :

- `donnerStatsSexe(idorg, idevent, idcourse, anneeevent)` — appel AJAX → `stats_sexe.php` → mise à jour du graphique ApexCharts barres groupées H/F par catégorie
- `donnerStatsSemaine(idorg, idevent, idcourse, anneeevent)` — appel AJAX → `stats_semaine.php` → mise à jour du graphique courbe d'inscriptions par semaine
- `donnerStatsCarte(...)` — appel AJAX → affichage des inscrits sur une carte MapLibre GL JS avec clustering par ville/département
- Toggle `#checkstat` : bascule entre le mode statistiques et le mode liste, masque/affiche les graphiques dynamiquement
- Persistance du filtre édition via paramètres GET (`?e=` , `?annee=` , `?c=`)

Page Détail Événement — `event_detail.php`

- Affichage des KPIs principaux : nb inscrits, gain KMS, gain total, comparatif édition précédente avec indicateurs de progression (flèche verte/rouge)
- Intégration du module statistiques complet (graphique sexe, graphique semaine, carte MapLibre)
- Section "Informations & Actualités" avec affichage de la plus récente
- Section dossiers litigieux (paiement manquant, certificat absent)
- Section documents à vérifier

- Navigation par édition (dropdown année)
 - Codes promo et extras avec statistiques d'utilisation
-

Page Détail Course — `course_detail.php`

- Mêmes KPIs et module statistiques que `event_detail.php`, au niveau d'une course précise
 - Affichage des informations complémentaires de la course
 - Section tarifs avec nb d'utilisations par tranche
 - Section extras avec réponses par catégorie
 - Accès rapide à la liste des inscrits filtrée sur cette course
-

Page Liste des Inscrits — `users_list.php`

- Tableau interactif **SimpleDataTables** avec pagination (10 / 50 / 100 / Tout), recherche en temps réel, tri par colonne
 - Filtre par événement + édition **ou** par course via deux dropdowns
 - Colonnes affichables/masquables avec persistance en **localStorage** : N° dossard, Nom, Club/Groupe, Genre, Date de naissance, Date d'inscription, Catégorie, Course, Payé, Licence/PPS
 - Mise en surbrillance de la ligne cliquée
 - **Export CSV** et **Export PDF** (jsPDF + autoTable) avec nom de fichier dynamique selon le filtre actif
 - Menu actions par participant : "Voir la fiche" et "Envoyer un mail"
 - Affichage du badge Payé / Non payé et Licence / Sans licence avec couleurs conditionnelles (vert/orange)
 - Gestion du numéro de dossard non attribué (affichage "N.A.")
-

Page Connexion — Impersonation Admin/Délégué

- Champ de recherche centralisée permettant à un **admin** ou **délégué** de rechercher un organisateur parmi tous les comptes
 - La recherche retourne tous les organisateurs correspondants avec leur identifiant
 - Sélection d'un organisateur → connexion en mode impersonation dans son espace sans connaître son mot de passe
 - Sécurité : vérification du rôle admin/délégué avant d'autoriser l'accès à cette fonctionnalité
-

Reset Password — AJAX

- Page de réinitialisation de mot de passe avec envoi d'un **code de reset par email**
 - Appel AJAX : l'utilisateur saisit son email, un `fetch()` envoie la demande sans rechargement de page
 - Génération d'un code temporaire côté PHP, stocké en session/BDD avec expiration
 - Retour JSON → affichage du message de confirmation ou d'erreur directement dans la page
-

Pages de consultation

- `home.php` — page d'accueil
 - `notifications.php` — page servant à la consultation des notifications
 - `course_detail.php` — page de détail d'une épreuve (statistiques, inscrits, extras, tarifs)
 - `event_detail.php` — page de détail d'un événement
 - `course_index.php` — liste des courses de l'organisateur
 - `users_list.php` — liste des participants
 - `stats.js` / `lib_esporg_stats.php` — module statistiques (semaine, sexe)
 - `upload_image.php` — upload d'images pour la description (limite 2 Mo)
-

Fichiers globaux & Infrastructure

- `lorenzo.js` — utilitaires globaux chargés partout : `elementExiste(id)`, `premiereLettreMaj(mots)`, gestion des notifications (lecture AJAX + décrémentation du compteur bulle)
 - `add.js` — protection "modifications non sauvegardées" via `beforeunload` sur tous les formulaires de création/modification
 - `upload_image.php` — endpoint d'upload d'image pour l'éditeur Quill (vérification type MIME + limite 2 Mo, retourne l'URL en JSON)
 - `head.php` / `navbar.php` / `script.php` — layouts partiels : balises meta, barre de navigation avec badges notifications/messages, scripts globaux
 - `editor.quill.js` — extension custom de Quill.js avec gestion du mode HTML source (bouton bascule éditeur ↔ code brut)
-

Points techniques notables

- Architecture **bibliothèque / actions** clairement séparée
- Toutes les requêtes SQL utilisent des **requêtes préparées PDO** (protection contre les injections)

- Le format de stockage des catégories d'équipe est une **chaîne encodée custom** (`Homme:HOM:M>=1.F<=5`) parsée côté PHP par `EspOrgGlobal__Donner_Equipe_Ctg`
- Le `UPDATE` du step3 cible **deux tables simultanément** (`course` + `courseparameqp`) via `INNER JOIN`
- Toutes les pages AJAX retournent du JSON avec `header('Content-Type: application/json')`
- Les statistiques géographiques utilisent les coordonnées GPS depuis la table `codepostauxville` pour alimenter la carte **MapLibre GL JS**